THE UNIVERSITY OF ZAMBIA

SCHOOL OF NATURAL SCIENCES

DEPARTMENT OF COMPUTER SCIENCE

NAME: PATRICIA MBOMA KASHWEKA

COMPUTER NUMBER: 2021527433

COURSE CODE: CSC 4505

LECTURER: MR MOFYA PHIRI

Introduction

For this project, a 3D surface of revolution was procedurally created and then included into a hierarchical humanoid model. After programmatically controlling the model's movements, Unity was used to deploy it in an augmented reality (AR) environment. This demonstrated the entire process, from mesh generation to deployment in an actual setting. The model's movements were controlled programmatically, and it was subsequently deployed in an augmented reality (AR) setting using Unity. This showcased a complete process from generating the mesh to deploying it in a real-world environment.

Objective

The main goal was to demonstrate proficiency in generating geometry with surface of revolution techniques, applying hierarchical transformations through parent-child relationships, and implementing character animations within Unity’s runtime environment.

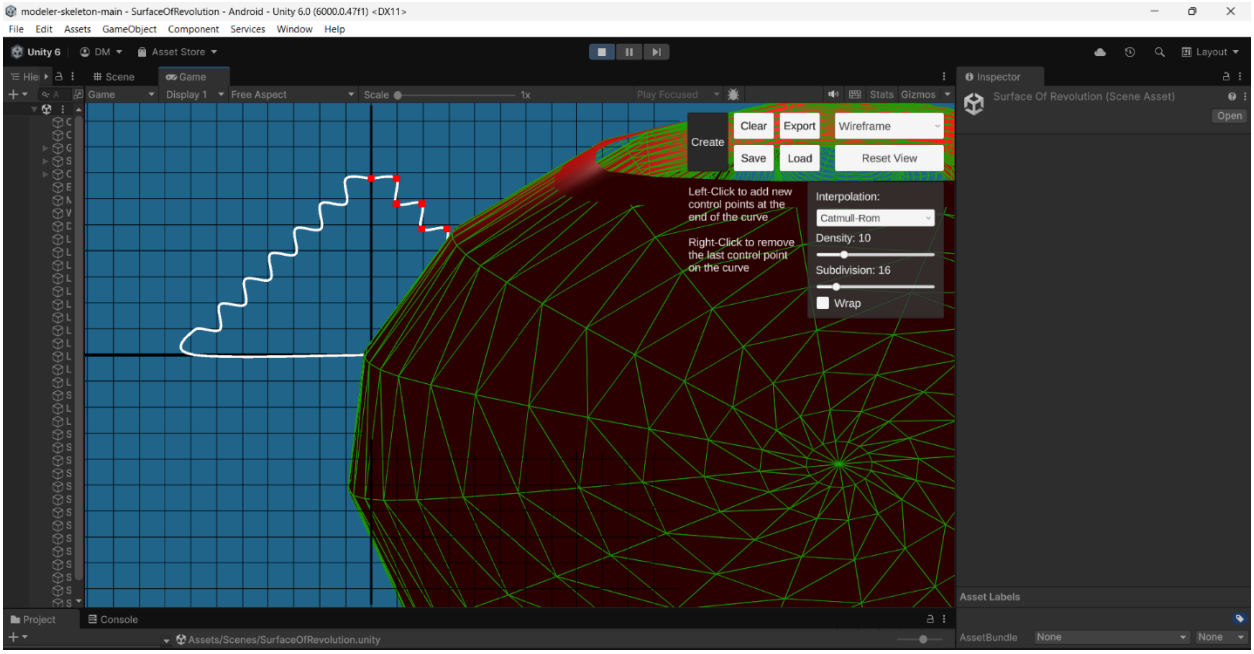
Surface of Revolution

To create the 3D mesh, a 2D profile curve was sampled and rotated incrementally around the Y-axis, producing concentric rings of vertices. These vertices were organized in a 2D array and connected using triangles to form the mesh surface.

Normals were calculated using cross products between adjacent vertices to ensure proper lighting and shading. UV coordinates were assigned based on the vertical progression along the profile (V-axis) and the rotational step (U-axis), enabling accurate texture mapping.

Several key decisions ensured mesh quality and visual accuracy. Clockwise winding was enforced to guarantee outward-facing normals. Mesh visualization tools were used to confirm normal orientation. To prevent UV distortion along the seam of the 360° rotation, edge vertices were duplicated. The final mesh was serialized and exported into a Unity directory for reuse across scenes.

Wireframe diagram



Hierarchical Model & Animation

The humanoid model was structured using a clear hierarchy. Each major limb or body segment was contained within an empty container GameObject. These containers acted as pivot points for local transformations, allowing for clean separation of motion and enabling modular animation.

The model hierarchy was structured as follows:

HumanoidModel

- BodyContainer
  - Body
  - HeadContainer
    - Head
    - summerhat
  - LeftHandContainer
    - LeftHand

RightHandContainer

    RightHand

LeftLegContainer

    LeftLeg

RightLegContainer

    RightLeg

Animation functionality was driven by three primary actions, each linked to a UI button. Button 1 triggered a full-body Y-axis rotation. Button 2 initiated a walk cycle, alternating arm and leg movement while adding vertical head motion for realism. Button 3 activated a looping jumping jack pose, involving lateral leg extensions and raised arms.

AR Deployment

Deployment to AR followed a structured approach. Unity's AR Foundation and ARCore XR plugins were installed via the Package Manager. The build platform was switched to Android, and core AR components AR Session and AR Session Origin were configured within the scene.

The humanoid prefab was integrated into the AR session hierarchy, scaled appropriately to fit real-world proportions, and deployed to an Android device using USB debugging. Several device-specific adjustments were made to ensure a smooth AR experience. These included tuning the near clipping plane, confirming camera and AR permissions, and ensuring that UI buttons remained responsive to touch inputs without interference from AR gestures.

Challenges & Solutions

Several technical challenges arose during development. Initially, some limbs rotated incorrectly due to misaligned pivot points. This was corrected by repositioning container GameObjects and aligning child meshes to the correct local axes.

Another issue involved animation conflicts. When triggering the walk cycle and custom animation simultaneously, the model behaved unpredictably. To resolve this, conditional logic was implemented to reset conflicting animation states, ensuring only one played at a time.

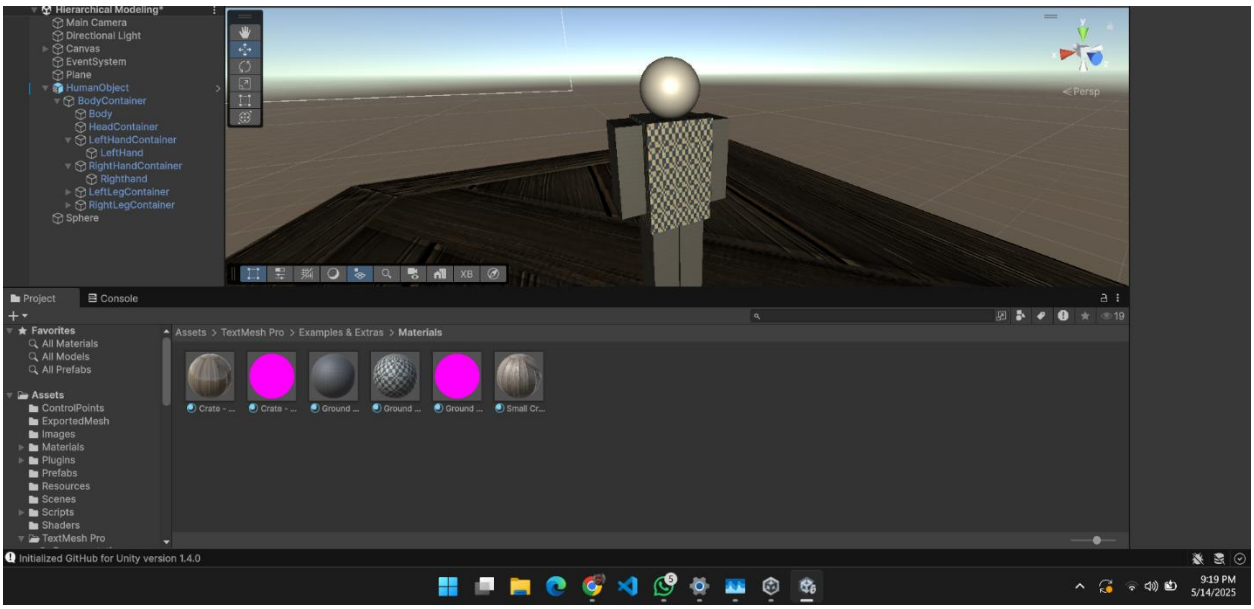
Early walk cycles lacked realism, as only the legs moved. To address this, coordinated arm swings and head bobbing were added to create a more natural locomotion effect. More broadly, fine-tuning animations proved time-consuming. Unity's animation preview tools were used to iteratively refine timing and movement accuracy.

AR deployment introduced its own complexities. At times, the model was either clipped or failed to render properly in the camera view. These issues were resolved by adjusting the AR camera's clipping planes and modifying the model's scale. Compatibility issues also arose due to version mismatches between the AR tools and the base Unity project. These were resolved by updating dependencies and reinstalling the AR Foundation packages.

Looking back, one critical improvement would be to define the hierarchy and pivot system earlier during mesh generation or in the modeling phase, rather than refining it later in Unity. This would streamline the animation process and reduce rework.

Evidence of Authentic Work

Screen shots to show progress of work, Time is present in the secreen shots



Development Workflow

The project progressed incrementally. Procedural mesh generation was developed first, followed by the assembly of the humanoid model into a reusable prefab. Animation logic and UI interactions were integrated in stages. After validating functionality on desktop, the final build was optimized and deployed to a mobile AR environment.

Progress Tracking:

Reference Materials

Key resources included a procedural surface of revolution tutorial on YouTube, Unity's AR Foundation documentation, and several community threads discussing prefab organization and animation state management.

Conclusion

This project underscored the importance of structured hierarchies and clean pivot logic in animation workflows. It also highlighted the added complexity introduced by AR development, particularly around scaling, hardware compatibility, and runtime constraints.

From a professional perspective, this work mirrors industry-standard pipelines in areas such as character rigging, procedural modeling, and AR deployment. It reinforced best practices in modular design, real-time performance optimization, and iterative debugging—critical skills in game development and interactive media engineering.